CSC302 Final Project

Elementary Number Theory in Lean,
starting from the Natural Number Game

Fall 2026

In the Natural Number Game you built the natural numbers from `0` and `succ`, and proved everything about $+$, $\times$, powers and $\leq$ from scratch. The final project continues on that foundation. You will choose a theorem from elementary number theory, formalize it in Lean on top of our Natural Number Game library, and explain it twice: once as mathematics on paper and once as Lean code.

You may, and are expected to, use AI tools to help write the Lean code. **You are not graded on whether you could have typed the proof yourself. You are graded on whether you understand every line of what you submit.** In the presentation you may be asked about any lemma in your code, and you should be able to say what it states, why it is true on paper, and how the Lean proof gets there.

Contents

1 Learning goals **2**

2 The starting library **2**

2.1 Setup and checking 2

2.2 Coming from the game 3

2.3 Practical notes 3

3 Topics **3**

4 Deliverables **9**

4.1 Lean code 9

4.2 Report (LaTeX, PDF) 9

4.3 Slides and presentation 10

5 Grading **10**

6 Advice **10**

1 Learning goals

By the end of the project you should be able to:

1. **Read Lean code.** Given a Lean proof (yours, the library's, or an AI's), state what each definition and theorem says in plain mathematics, and follow each tactic step.
2. **Explain the mathematics.** Give a complete paper proof of your theorem at the level of a good textbook.
3. **Connect the two.** Point to the place in the Lean code where each step of the paper proof happens, and explain where Lean needed more detail than the paper proof did.

2 The starting library

The project repository contains a folder `lean/`: a stand-alone Lean 4 project with **no Mathlib**. You build on this library, not on Mathlib and not on Lean's built-in `Nat` theorems.

Module	What it gives you
`FinalProject/NNG/*`	`MyNat` (written $\mathbb{N}$), $+$, $\times$, `^`, $\leq$, and every theorem from the game's worlds under the same names (`add_comm`, `mul_left_cancel`, `le_antisymm`, ...)
`Modulo/Order`	$<$, trichotomy, decidability of $\leq$ and $<$, **strong induction**, well-founded recursion on $\mathbb{N}$
`Modulo/Divisibility`	$a \mid b$ means $\exists c,\ b = a \cdot c$, with its basic API
`Modulo/DivMod`	`a / n` and `a % n`, which compute (`#eval (17 : ℕ) % 5` prints `2`); $n \cdot (a/n) + a\%n = a$; $a\%n < n$ for $n \neq 0$; uniqueness of quotient and remainder
`Modulo/Congruence`	$a \equiv b$ [MOD n] and its compatibility with $+$, $\times$, powers
`Modulo/Prime`	the definition `Prime p`: $2 \leq p$ and the only divisors of p are 1 and p
`Tactic/Ring`	lets the `grind` tactic prove polynomial identities in $\mathbb{N}$ (see Section 4.1)

One difference from the game. There, $+$, $\times$ and powers were given by axioms. Here they are defined by recursion, so the six "axioms" `add_zero`, `add_succ`, `mul_zero`, `mul_succ`, `pow_zero`, `pow_succ` are now theorems with the same names, and Lean can *compute*. This is what makes algorithm projects (Euclid's algorithm, the Chinese remainder theorem) possible.

No subtraction. As in the game, the library has no subtraction. A statement that would normally say $n \mid b - a$ is phrased as $\exists k,\ b = a + nk$. Getting used to this is part of the project.

2.1 Setup and checking

1. Install Lean with `elan`, as for the game outside the browser. The project pins its own Lean version in `lean/lean-toolchain`; there is no Mathlib download.
2. Each topic has a starter file in `lean/Projects/` (for example `A1_Euclid.lean`) with the definitions and the statements to prove, each ending in `sorry`. Copy it and work in the copy.

Your file imports `FinalProject` and nothing else (Track F: only `FinalProject.NNG`).

3. Check your file with `lean/scripts/check.sh Projects/<your file>`. The script compiles the library and your file and reports any error or `sorry`.

2.2 Coming from the game

The game used a few custom tactics. Standard Lean, and therefore AI-written code, spells some of them differently.

In the game	In this project
`induction n with d hd`	`induction n with | zero => ... | succ d hd => ...`
`cases h with c hc`	`obtain ⟨c, hc⟩ := h`
`use 37`	`use 37` (provided by the library)
`rw [h]`	the same; `rw` also tries `rfl` afterwards
`decide`	`decide` (works because arithmetic computes)

2.3 Practical notes

- **Numbers are unary.** `MyNat` stores 5 as `succ (succ (succ (succ (succ 0))))`. Plain `decide` handles numbers up to about 50; beyond that it stops with "maximum recursion depth". Use `decide +kernel` instead (it checks `Prime 97` or `5 ∣ 720` in a moment), and `#eval` for exploring.
- **Functions defined by well-founded recursion** (with `termination_by`, such as Euclid's `gcd`) run under `#eval`, but `decide` and `rfl` cannot unfold them. In proofs, rewrite with the function's equation (`rw [gcd]`). Functions defined by structural recursion, like `%`, have no such problem.
- **Proof by contradiction.** Without Mathlib there is no `by_contra` or `push_neg`. Use `Classical.byContradiction fun h => ...`, or `by_cases h : P` and handle both cases.
- **Track F files** import only `FinalProject.NNG`; everything else in this table is then unavailable.

3 Topics

Pick one topic, or propose your own and get it approved first. Stars rate the difficulty *in this library*, where you have no Mathlib and build what you need yourself.

Size. Your Lean file should be about **300 to 400 lines**, comments included. After each title is the size of a complete solution. "(m)" means it was *measured*: we wrote a reference solution with an AI assistant, in the style we expect from you. The other sizes are estimates. Where a topic is smaller than 300 lines, its description says which **extension** brings it up to size; do the extension. The extension is part of the topic.

"Needs X" means the topic uses results of topic X. Prove those as the first part of your file; the size already includes them. If only part of X is needed, the topic says which part.

	Topic	Difficulty	Lines
F1	Strong induction and well-ordering	★★	350
F2	The division algorithm	★★	300–350
F3	The integers	★★★	350–450
F4	Finite sums and the binomial theorem	★★★	350–450
F5	The pigeonhole principle	★★★	300
A1	Euclid's algorithm and its extension	★★	290 (m)
A2	Cancellation and modular inverses	★★★	350
A3	The Chinese remainder theorem	★★★	308 (m)
A4	Fibonacci numbers and gcd	★★★★	450–550
B1	There are infinitely many primes	★★	350
B2	Euclid's lemma and unique factorization	★★★	323 (m)
B3	Irrationality of square roots	★★	300–350
C1	Divisibility rules	★★	300
C2	Fermat's little theorem	★★★★	450–550
C3	Wilson's theorem	★★★★★	600+
D1	Sums of two squares	★★	320
D2	Euler's four-square identity	★★★	300–400
D3	Squares modulo a prime	★★★	350
D4	The key congruence	★★★★★	600+
D5	Reduction to primes; three squares	★★	350

Track F: Foundations (extend the game yourself)

These topics rebuild part of our library from the game alone. Your file imports `FinalProject.NNG` (the game layer) *instead of* `FinalProject`, so the library's version of what you prove is not available. You may read it, but your proofs must be your own.

F1. Strong induction and the well-ordering principle ★★ *about 350 lines; 260 (m) without the extension*

Ordinary induction steps from n to $n+1$. Strong induction may use *all* smaller cases.

- Define $a < b$ as $a + 1 \le b$ and prove the facts you need about it.
- `strong_induction : (∀ n, (∀ m, m < n → P m) → P n) → ∀ n, P n`, proved from ordinary induction. Hint: prove $\forall n,\ \forall m < n,\ P\,m$ by ordinary induction on n.
- Well-ordering: if $P\,n$ holds for some n, there is an n with $P\,n$ such that $n \le m$ for every m with $P\,m$.
- No infinite descent: if every n with $P\,n$ has some $m < n$ with $P\,m$, then P never holds.
- Application: every $n \ge 2$ has a prime divisor; take the least divisor $d \ge 2$ of n and prove that d is prime.
- *Extension:* prove that the three principles are equivalent: each of strong induction, well-ordering and no infinite descent implies the other two, using only ordinary induction in between.

F2. The division algorithm ★★ *about 300–350 lines (estimate)*

The theorem behind `%`: for $n \neq 0$ and any a there are q, r with $a = nq + r$ and $r < n$, and they are unique. Define `div` and `mod` as functions, prove they return exactly this q and r when $n \neq 0$ (you choose what they do for $n = 0$), and check examples with `#eval`. *Extension:* the rules of remainder arithmetic, $(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$ and the same for products.

F3. The integers ★★★ *about 350–450 lines (estimate)*

The game has no negative numbers. Build them: an integer is a pair (a, b) of natural numbers, thought of as $a - b$, where (a, b) and (c, d) count as equal when $a + d = b + c$. Define $+$, $\times$ and negation, prove they respect this equality, and prove the ring laws (commutativity, associativity, distributivity, $x + (-x) = 0$). This is how mathematicians construct $\mathbb{Z}$ from $\mathbb{N}$.

F4. Finite sums and the binomial theorem ★★★ *about 350–450 lines (estimate)*

Define sum $n\ f = f(0) + \cdots + f(n-1)$ by recursion. Prove $2\,(0 + 1 + \cdots + n) = n(n+1)$ and that the first n odd numbers sum to n^2; define binomial coefficients by Pascal's rule and prove the binomial theorem $(a + b)^n = \sum_k \binom{n}{k} a^k b^{n-k}$. Since there is no subtraction, part of the work is to state it without $n - k$, for example as a sum over pairs $i + j = n$.

F5. The pigeonhole principle ★★★ *about 300 lines (estimate)*

If $n+1$ numbers $f(0), \ldots, f(n)$ all lie below n, two of them are equal: `(∀ i, i ≤ n → f i < n) → ∃ i j, i < j ∧ j ≤ n ∧ f i = f j`. Apply it: among any $n+1$ numbers, two are congruent modulo n (for $n \geq 1$); and for every $n \geq 1$, one of the numbers $1, 11, 111, \ldots$ (with at most $n + 1$ ones) leaves the same remainder modulo n as an earlier one. Since this is Track F, "congruent" here means "differ by a multiple of n", and you prove yourself that every number has a remainder below n.

Track A: Algorithms (gcd and its uses)

A1. Euclid's algorithm and the extended Euclidean algorithm ★★ *290 lines (m)*

Define `gcd` by Euclid's recursion $\gcd(0, b) = b$, $\gcd(a, b) = \gcd(b \bmod a, a)$, prove it terminates and that it is the greatest common divisor. Then define the extended algorithm, which also returns Bézout coefficients, and prove its output correct.

- `gcd a b ∣ a`, `gcd a b ∣ b`, and `d ∣ a → d ∣ b → d ∣ gcd a b`.
- Bézout without negative numbers: if $a \neq 0$ there are x, y with $ax = by + \gcd(a, b)$. (For $a = 0$, $b = 1$ there are none.)
- The extended algorithm is correct: state precisely what its output satisfies (the form of the output is your design) and prove it.
- `#eval` your algorithms on examples and show the output in your slides.

Hint. Without negative numbers the Bézout equation changes sides at every step of the recursion: from the pair $(b \bmod a, a)$ you get $bx = ay + g$, the wrong way round for (a, b). So the extended algorithm must keep track of both forms, $ax = by + g$ and $by = ax + g$, and the final step converts one into the other. Termination: copy the `termination_by`/`decreasing_by` pattern from the library's `Modulo/Order`.

A2. Cancellation in congruences and modular inverses ★★★ *about 350 lines (estimate)* (needs `gcd` and Bézout from A1)

When may you divide both sides of a congruence by a number?

- If $\gcd(c, n) = 1$ and $ac \equiv bc \pmod{n}$, then $a \equiv b \pmod{n}$.
- c has an inverse modulo n (some x with $cx \equiv 1 \pmod{n}$) if and only if $\gcd(c, n) = 1$.
- A counterexample in Lean showing cancellation fails without the hypothesis: $2 \cdot 2 \equiv 2 \cdot 5 \pmod{6}$ but $2 \not\equiv 5 \pmod{6}$.
- Extension: $ax \equiv b \pmod{n}$ has a solution if and only if $\gcd(a, n) \mid b$.

A3. The Chinese remainder theorem ★★★ *308 lines (m)* (needs A1)

- Existence: if $\gcd(m, n) = 1$, then for all a, b there is x with $x \equiv a \pmod{m}$ and $x \equiv b \pmod{n}$. Write a function that computes such an x and `#eval` it (Sunzi's problem: $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$).
- Uniqueness: two solutions are congruent modulo mn. Key lemma: if $m \mid k$, $n \mid k$ and $\gcd(m, n) = 1$, then $mn \mid k$.

A4. Fibonacci numbers and gcd ★★★★ *about 450–550 lines (estimate)* (needs A1)

Prove $\gcd(F_m, F_n) = F_{\gcd(m,n)}$. Along the way: consecutive Fibonacci numbers are coprime, and $F_{m+n+1} = F_{m+1}F_{n+1} + F_mF_n$.

Track B: Primes

B1. There are infinitely many primes ★★ *about 350 lines; 172 (m) without the extension*

- Every $n \geq 2$ has a prime divisor (strong induction).
- Define the factorial and prove $d \mid n!$ for $1 \leq d \leq n$.
- Euclid's theorem: for every n there is a prime p with $n < p$.
- *Extension:* (a) a function `minFac` computing the smallest divisor ≥ 2, with a proof that it is prime and `#eval` examples; (b) there are infinitely many primes $\equiv 3 \pmod{4}$. For (b): a product of numbers $\equiv 1 \pmod{4}$ is $\equiv 1 \pmod{4}$, so every $n \equiv 3 \pmod{4}$ has a prime divisor $\equiv 3 \pmod{4}$; apply this to the number N with $N+1 = 4 \cdot n!$, i.e. $4 \cdot n! - 1$ (why not $4 \cdot n! + 3$?).

B2. Euclid's lemma and unique factorization ★★★ *323 lines (m)* (needs `gcd`, `gcd_dvd_left/right` and Bézout from A1)

- Euclid's lemma: if p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.
- Define the product of a list, `prod : List` $\mathbb{N} \to \mathbb{N}$.
- Existence: every $n \geq 1$ is the product of a list of primes.
- Uniqueness: two lists of primes with the same product are permutations of each other (Lean's `List.Perm`).

Hint. Useful lemmas from Lean's list library: `List.append_of_mem`, `List.perm_middle`, `List.Perm.cons`, `List.Perm.trans` and `List.mem_cons`.

B3. Irrationality of square roots ★★ *about 300–350 lines (estimate)* (needs Euclid's lemma (B2) for the extension)

- $\sqrt{2}$ is irrational: there are no a, b with $b \neq 0$ and $a^2 = 2b^2$ (parity and infinite descent).
- *Extension:* for any prime p, there are no a, b with $b \neq 0$ and $a^2 = p\,b^2$.

Track C: Congruences at work

C1. Divisibility rules ★★ *about 300 lines (estimate)*

Represent a number by its list of decimal digits. Prove that n is congruent to its digit sum modulo 9 (hence modulo 3), and the rule for 11 in subtraction-free form: n plus the sum of the digits in odd positions is congruent modulo 11 to the sum of the digits in even positions.

C2. Fermat's little theorem ★★★★ *about 450–550 lines (estimate)* (needs Euclid's lemma (B2))

For a prime p, $a^p \equiv a \pmod p$. Route: binomial coefficients, $p \mid \binom{p}{k}$ for $0 < k < p$, hence $(a+1)^p \equiv a^p + 1 \pmod p$, and induction on a.

C3. Wilson's theorem ★★★★★ *600 lines or more (estimate)* (needs A2)

For a prime p, $(p-1)! \equiv -1 \pmod p$. Without subtraction: if $p = k+1$ is prime, then $p \mid k! + 1$. The proof pairs each number with its inverse; making that pairing precise is the hard part. For ambitious teams only.

Track D: Toward the four-square theorem

Lagrange's theorem (1770): **every natural number is a sum of four squares.** The whole proof is too long for one project in our library, so it is cut into pieces. Each piece is a complete theorem. A piece that needs another one takes it as a *hypothesis* in its statement, so different teams can work in parallel; together they cover most of the proof.

Write `IsSum2 n`, `IsSum3 n`, `IsSum4 n` for "n is a sum of two (three, four) squares", e.g. `IsSum2 n` is $\exists a\, b,\ n = a \cdot a + b \cdot b$. The classical proof:

1. Euler's identity: a product of two sums of four squares is a sum of four squares (D2).
2. So it is enough to treat primes, since every number is a product of primes (D5).
3. For an odd prime $p = 2k+1$ the $k+1$ squares $0^2, 1^2, \ldots, k^2$ are different modulo p (D3), and a pigeonhole argument gives $a, b \leq k$ with $p \mid a^2 + b^2 + 1$ (D4). So some multiple mp with $m < p$ is a sum of four squares.
4. Descent: if mp is a sum of four squares with $1 < m < p$, so is $m'p$ for some $m' < m$. Repeating, p itself is a sum of four squares. (Not offered as a project: it is the hardest step.)

D1. Sums of two squares ★★ *about 320 lines; 261 (m) without the obstruction*

- `IsSum2 m` → `IsSum2 n` → `IsSum2 (m * n)`. On paper, $(a^2+b^2)(c^2+d^2) = (ac+bd)^2 + (ad-bc)^2$. Without subtraction, split into the cases $ad \le bc$ and $bc \le ad$. Prove the identity in each case *twice*: once with `grind`, once by hand with `rw`, and explain what `grind` did.
- A function computing the pair for mn from the pairs for m and n, with `#eval` ($5 \cdot 13 = 65 = 1^2+8^2 = 4^2+7^2$); and: if n is a sum of two squares, so is n^k.
- No number $\equiv 3 \pmod 4$ is a sum of two squares.

D2. Euler's four-square identity ★★★ *about 300–400 lines (estimate)*

`IsSum4 m` → `IsSum4 n` → `IsSum4 (m * n)`. Four of the eight products in the identity enter with a minus sign, so in $\mathbb{N}$ you need case splits on which of two products is larger, or the integers from F3. Include a function computing the four squares and `#eval` examples.

D3. Squares modulo a prime ★★★ *about 350 lines (estimate)* (needs Euclid's lemma (B2))

For a prime p: $x^2 \equiv y^2 \pmod p$ implies $x \equiv y$ or $x + y \equiv 0 \pmod p$. Deduce that if $p = 2k+1$, then for $i < j \le k$ the squares i^2 and j^2 are not congruent modulo p.

D4. The key congruence ★★★★★ *600 lines or more (estimate)* (needs D3, F5, as hypotheses)

For an odd prime $p = 2k+1$ there are $a, b \le k$ with $p \mid a^2 + b^2 + 1$. The values a^2 and the values "the residue r with $r + b^2 + 1 \equiv 0$" are $2k+2$ numbers in $2k+1$ residue classes, and within each family they are distinct by D3. For ambitious teams.

D5. Reduction to primes, and why three squares are not enough ★★ *about 350 lines (estimate)*

- Assuming Euler's identity (D2) and that every prime is a sum of four squares, prove that every natural number is a sum of four squares (strong induction; every $n \ge 2$ has a prime divisor).
- No number $\equiv 7 \pmod 8$ is a sum of three squares; so four is the right number. Measured: 183 lines (m) for this item and the next.
- No number of the form $4^a(8b+7)$ is a sum of three squares.

Hint. Replace each number by its remainder (a square's remainder depends only on the number's remainder), then let `decide` check the finitely many cases.

Your own topic

Good sources: the parity and divisibility chapters (3 and 7) of *The Mechanics of Proof*, and any elementary number theory textbook. Bring the precise statement and a paper proof to the instructor before you start. Avoid statements that need negative numbers, rationals or reals unless you have a subtraction-free reformulation.

4 Deliverables

Submit all three. They describe the same project from three angles.

4.1 Lean code

- One file in `lean/Projects/`, grown from your topic's starter file and importing only `FinalProject` (Track F: only `FinalProject.NNG`). A long project may be split into several files under `Projects/<YourTopic>/`.
- **It compiles** with `scripts/check.sh`, with no `sorry`, `admit`, `axiom` or `native_decide`.
- The file ends with `#print axioms` for your main theorem. The output may contain only `propext`, `Classical.choice` and `Quot.sound`, or nothing.
- About 300–400 lines, more for harder topics. Quality matters more than length; do not pad.
- **Not allowed:** importing Mathlib, converting to Lean's built-in `Nat` to use its theorems or `omega`, or changing the library. If you need a general fact the library lacks, prove it in your own file.
- `grind` may be used *only* to close an equation between polynomial expressions, such as $(a + b)^2 = a^2 + 2ab + b^2$ or an algebraic identity in D1 or D2. The library's `Tactic/Ring` module makes this work by handing `grind` the game's theorems (`add_comm`, `mul_assoc`, distributivity, ...); be ready to explain that. Do not use `grind` for inequalities, divisibility or logic.
- Every definition and theorem gets a short comment saying in plain words what it states. The main theorem gets a docstring with the paper statement.

4.2 Report (LaTeX, PDF)

Use the template `templates/report.tex`. You may ask an AI to help with LaTeX formatting; the content must be yours. About 6–10 pages:

1. **Introduction.** The theorem, why it matters, a little history.
2. **Paper proof.** A complete proof for a reader who knows the course material. State every lemma you use.
3. **Lean formalization.** Your definitions and why you chose them; the main theorem in Lean, explained symbol by symbol; a walk-through of the key lemmas.
4. **Correspondence table.** Each step of the paper proof next to the Lean lemma that carries it out (the template has a table for this).
5. **What Lean made you do.** Places where the paper proof says "clearly" and Lean needed real work; edge cases such as 0 and 1; how the lack of subtraction changed statements.
6. **AI use.** Which tools you used and for what; one example where AI output was wrong or unhelpful, and how you found out.

4.3 Slides and presentation

Use the template `templates/slides.tex` (Beamer), or any slide tool exported to PDF. About 10–15 slides for a 12–15 minute talk: the theorem with a small example (`#eval` output is a nice touch), the idea of the paper proof, the Lean statement explained, one or two key lemmas shown in Lean and explained line by line, and what was hard.

After the talk there is a short question period. Expect to be pointed at a lemma in your code and asked what it says, why it is true, and how the proof works.

5 Grading

Component	Weight	What we look for
Lean code	30%	compiles, no `sorry`, statements say what the report claims, readable comments
Report	30%	correct and complete paper proof; clear explanation of the code; the correspondence table
Presentation and questions	40%	clear talk; can explain any lemma in the code both on paper and in Lean

A correct project on a ★★ topic, explained well, earns full credit. Harder topics earn more room for error, not automatic extra points.

6 Advice

- **Read the statement before the proof.** The most common failure is a Lean theorem that is proved but does not say what you think: a hypothesis that can never hold, $\leq$ where you meant $<$, a missing $n \neq 0$. Test your main statement on small cases with `decide` or `#eval`.
- **Start from the paper proof.** Write it first; then ask AI to formalize one lemma at a time and check that each one compiles before moving on.
- **When AI is stuck** it often repeats the same attempt. Break the step into smaller lemmas yourself, following your paper proof.
- **Zero is always the edge case.** $a\%0 = a$, $a/0 = 0$, $0 \mid a$ only for $a = 0$, and $\gcd(0, 0) = 0$. Decide early how your statements treat zero.
- **Commit often,** and keep your file compiling at every commit.